

ARTEMIS System Design

System Architecture

Revision 2

FAST NUCES Final-Year Project **F26-008**

Adaptive Reasoning, Task Execution and Multi-agent Intelligence System

Team member	Roll number
Hashir Rauf	23L-2572
Hira Khalid	23L-2594
Doureesha Batool	23L-2651

Supervisor: Mr. Mubasher Hussain

Internal design document. Not for external distribution.

Compiled September 9, 2026

Contents

1 Purpose and Status of This Document	1
2 Architectural Thesis	1
2.1 Degradation contract	1
2.2 Non-goals	2
3 Layered Overview	2
3.1 Why two planes rather than two further layers	3
3.2 Component inventory	3
4 The Request Lifecycle	3
5 Layer 3: The AI Gateway	4
5.1 Model Router	4
5.2 Context Engine	5
5.3 Policy Engine	5
5.4 Agent Planner	6
6 Layer 4: Capability Services	6
6.1 Workspace Broker	6
6.2 Undo Journal	7
7 Layer 5: The Data Plane	7
7.1 Memory architecture	7
7.2 The disclosure panel	8
8 Layer 6: The Remote Access Plane	8
8.1 Pairing	9
8.2 Constraints on remote sessions	9
9 The Security and Trust Plane	9
9.1 Prompt injection	9
9.2 Threat model	9
10 The Execution and Performance Plane	9
11 Process and Deployment Topology	10
12 Traceability Between Architecture and Research	11
13 Build Sequence	11
14 Open Questions	11
15 Summary of Changes from Revision 1	13

1 Purpose and Status of This Document

This document specifies the proof-of-concept architecture for ARTEMIS. It supersedes the outline architecture recorded in the prototype development plan, which is referred to throughout as Revision 1.

Revision 1 described the system as six stacked boxes with an AI gateway at the centre. That description was adequate for communicating intent but inadequate as an engineering specification, for three reasons. The gateway was a single component responsible for routing, context assembly, memory, planning, policy and dispatch simultaneously, which is neither testable nor separable. Security appeared as a labelled box at the foot of the diagram rather than as a property of the whole system. And the remote-access facility was drawn inside the core stack, which weakened the claim that ARTEMIS works without a network.

Revision 2 addresses all three. It decomposes the gateway into four cooperating components behind a single execution chokepoint, promotes security and performance to cross-cutting planes, and draws the remote-access facility outside the core so that its removal provably changes nothing about local operation. It also introduces one component Revision 1 did not name at all, the Policy Engine, which is described in Section 5.3 and which is the mechanism by which this project's central research finding becomes enforced structure rather than stated intention.

Section 15 lists every change from Revision 1 with its rationale.

2 Architectural Thesis

Every structural decision in this document follows from a single sentence.

Principle P1. *Design thesis*

ARTEMIS is local-first, cloud-optional and remotely accessible, but never cloud-dependent for core operation, and never in possession of scope the user has not explicitly granted.

This is not a deployment preference. It is the architectural encoding of the project's central research hypothesis: that restricting an assistant's scope increases user trust and adoption without meaningfully reducing its usefulness. If that hypothesis is to be tested honestly, the restriction has to be structural rather than behavioural. It must be impossible by construction, not merely discouraged by instruction. A boundary that the system could cross but chooses not to is a policy; a boundary the system has no code path across is an architecture. This document is concerned throughout with the second kind.

2.1 Degradation contract

The correctness of a local-first system is defined by how it behaves as capabilities are withdrawn from it. The rows of Table 1 are acceptance criteria, each one testable, not aspirations.

Table 1: Required behaviour under withdrawal of each external capability

Withdrawn capability	Required behaviour
Internet connectivity lost	Full local operation continues. Only cloud model calls, web search and mail sending are disabled, and each fails as an explicit, retryable queued action rather than a silent drop.
All cloud model APIs unavailable	Falls back to the configured local model. Features exceeding local capability degrade to a stated limitation, not an error.
Local model unavailable	Falls forward to the cloud chain, if and only if the user has enabled cloud fallback for that workspace.
Remote device disconnects	The host continues uninterrupted. Approvals in flight revert to pending and surface on the host interface.
Cloudflare edge unreachable	Local operation is entirely unaffected, because the remote plane is not on the execution path.
Workspace not explicitly granted	ARTEMIS cannot see it. This is not a refusal to act; the path is not resolvable through the Workspace Broker at all.

2.2 Non-goals

An architecture is defined as much by what it refuses as by what it provides. The following are permanently out of scope and are enforced, not merely avoided.

- **No hosted user data.** Documents, embeddings, session history, memory, credentials and audit logs never leave the host machine. The only outbound telemetry is anonymous, opt-in and schema-fixed diagnostic logging.
- **No autonomous outward action.** No message is sent, no web request issued and no file modified without passing an approval gate.
- **No ambient surveillance.** There is no background browsing, screen capture, keystroke logging or always-listening recording. A voice session is explicitly armed by the user and visibly indicated while active.
- **No self-modifying execution.** The agent cannot write and then run arbitrary code against the host as a step in its own plan.

3 Layered Overview

The system is organised as six layers on the execution path, with two planes crossing all of them. Figure 1 shows the whole arrangement.

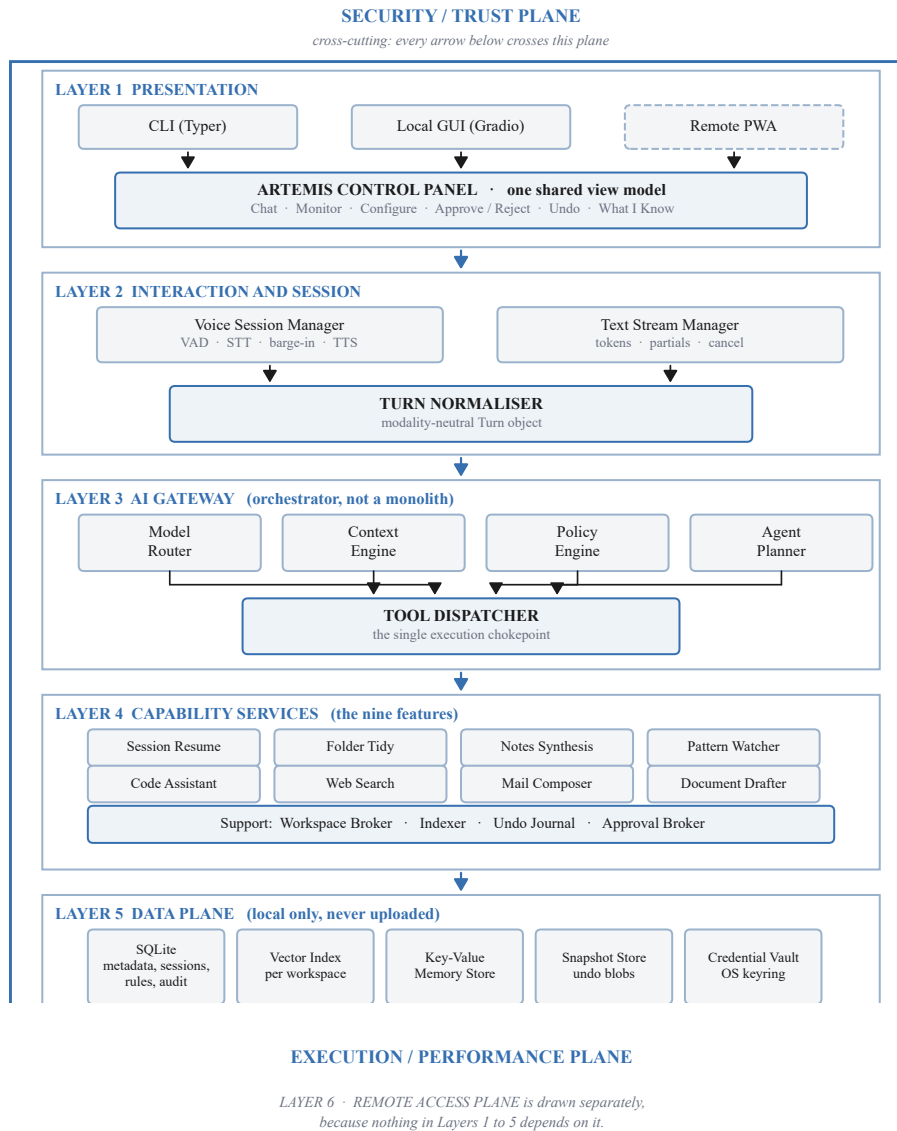


Figure 1: The ARTEMIS layered architecture. Layers 1 to 5 form the local execution path. The security and performance planes are drawn as enclosing bands rather than as layers because they are properties every layer must hold. Layer 6 is drawn separately, in Figure 13, because nothing in Layers 1 to 5 depends on it.

3.1 Why two planes rather than two further layers

Security and performance are not stages through which a request passes. They are properties that every stage must hold simultaneously. Drawing them as layers would imply that a request can be past security, which is precisely the reasoning error that produces confused-deputy defects: a component trusts an input because some earlier component is assumed to have checked it. Drawing them as planes states the opposite, that every arrow in the figure crosses both, and that no component may assume a check has already happened.

3.2 Component inventory

Table 2 lists all eighteen components. The final column records the single constraint each component exists to hold, which is the property whose violation would break the design thesis of Section 2.

Table 2: Component inventory, with the invariant each component holds

#	Component	Layer	Responsibility	Must never
1	Command-line interface	1	Scriptable control surface	Bypass the approval broker
2	Graphical interface	1	Visual control panel	Bind to a non-loopback interface
3	Control panel view model	1	Single source of interface truth	Hold state the backend does not own
4	Voice Session Manager	2	Detection, transcription, speech output	Record when not explicitly armed
5	Text Stream Manager	2	Token streaming and cancellation	Buffer past a cancellation
6	Turn Normaliser	2	Modality-neutral turn object	Leak modality downstream
7	Model Router	3	Model selection and failover	Send workspace data to a disabled tier
8	Context Engine	3	Bounded, cited context assembly	Include content outside the workspace
9	Policy Engine	3	Risk classification and gating	Be consulted after execution
10	Agent Planner	3	Multi-step plan construction	Execute anything itself
11	Tool Dispatcher	3	Sole execution chokepoint	Accept a call lacking a verdict
12	Capability services	4	The nine product features	Touch the filesystem directly
13	Workspace Broker	4	Path resolution and confinement	Resolve a path outside a grant
14	Indexer	4	Incremental embedding and indexing	Block the interactive path
15	Undo Journal	4	Snapshots and inverse operations	Record an action after the fact
16	Approval Broker	4	Pending actions across surfaces	Auto-approve on timeout
17	Data plane stores	5	Local persistence	Serialise outside the host
18	Remote access plane	6	Authenticated relay	Store or read plaintext user data

4 The Request Lifecycle

Figure 2 shows the single most important path in the system. Every request, whether spoken or typed, and whether raised locally or from a paired remote device, traverses exactly this sequence. There are no shortcuts and no privileged entry points.

Five invariants hold along this path, and each is directly testable.

1. The Policy Engine is consulted before the Tool Dispatcher, never after.
2. The Undo Journal snapshots before mutation, never after.
3. An audit entry is written even when the action is rejected, so that the log records what was proposed as well as what was performed.
4. A tool call carrying no policy verdict is rejected by the dispatcher.
5. Voice and text turns are indistinguishable from the planner onward.

The fifth invariant is the mechanism behind the voice-parity claim discussed in Section 6, and it is worth stating plainly why it matters. If the voice path were a separate implementation, then every approval gate would exist twice, and the two copies would drift. Because modality is erased before the planner, the gates exist once and voice inherits them without further work.

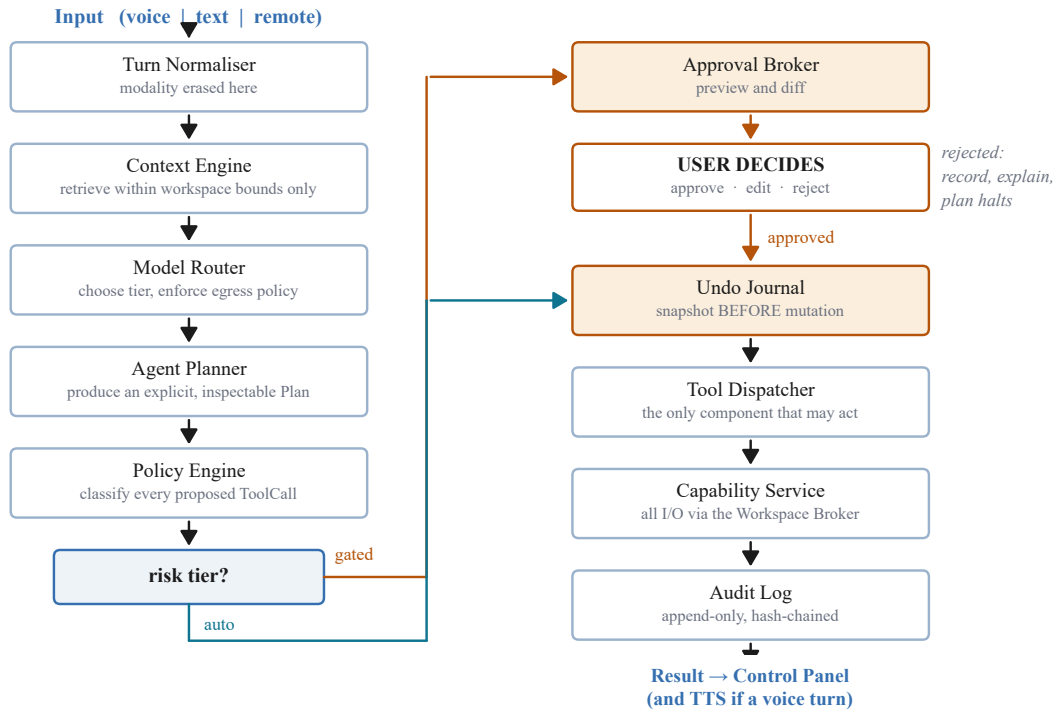


Figure 2: The canonical request lifecycle. Modality is erased at the Turn Normaliser, so every stage below it is identical for voice and for text. The gated branch returns to the trunk beneath the Undo Journal, so that an approved action is snapshotted before it is dispatched.

5 Layer 3: The AI Gateway

Revision 1 assigned routing, context assembly, memory, planning, policy and dispatch to a single gateway component. Revision 2 separates these into four components that cooperate through one shared execution chokepoint, as shown in Figure 3.

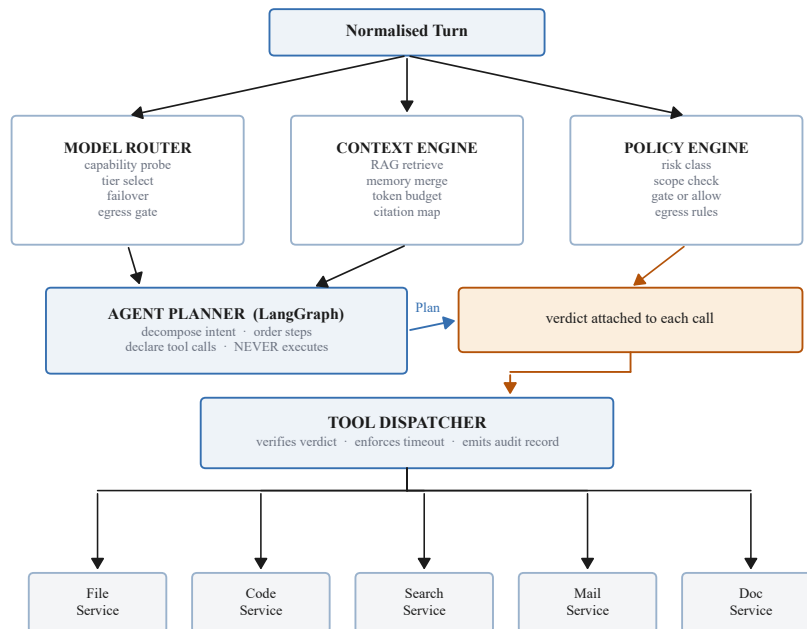


Figure 3: Internal structure of the AI gateway. The planner produces a plan but never executes it; the Tool Dispatcher executes but never decides. Separating the two is what makes the pause before any workspace-changing call structural rather than conventional.

5.1 Model Router

The Model Router owns tier selection and, more importantly, data-egress enforcement. It is the only component that knows a given workspace's cloud policy, which makes it the correct place to block egress. Placing that

check in the tool layer would be too late, and placing it in the prompt would not be a check at all.

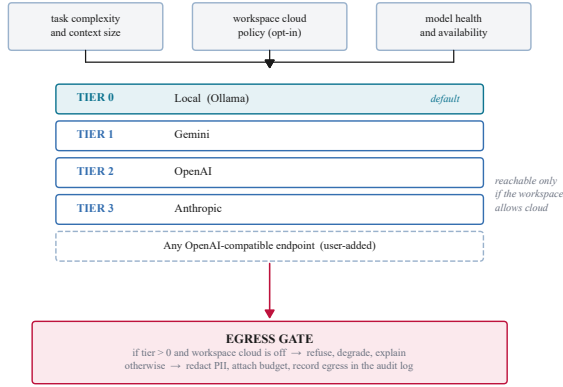


Figure 4: Tier selection and the egress gate. Tiers above zero are reachable only when the active workspace permits cloud use; otherwise the router degrades and explains rather than silently succeeding.

The fallback order from the proposal is retained: local models first, then Gemini, then OpenAI, then Anthropic. Any endpoint offering an OpenAI-compatible interface can be registered as a further tier through configuration alone, so adding a provider requires no code change. Failover is bounded at one retry per tier before advancing, so a workspace that has disabled cloud access has exactly one tier and fails cleanly rather than hanging through a chain of unreachable providers.

5.2 Context Engine

The Context Engine assembles the working context for a request and enforces the bounded scope hypothesis on the read side. The Workspace Broker, described in Section 6.1, enforces the same boundary on the write side.

The citation map is what makes citation-grounded synthesis verifiable rather than merely plausible. It is carried through to the rendered answer, so a claim that cannot be traced back to a specific chunk, file and location is marked as unsupported rather than presented alongside supported claims. This directly addresses the hallucination and citation-accuracy themes of the literature review.

5.3 Policy Engine

The Policy Engine is the most significant addition in this revision. It converts the research finding that users want approval-first behaviour, configurable to their own tolerance, from an interface convention into an enforced architectural component.

Every proposed tool call is classified into exactly one of four tiers.

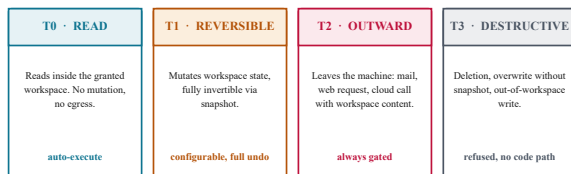


Figure 6: The four risk tiers. Only the reversible tier is user-configurable; outward actions are always gated, and destructive actions are refused structurally.

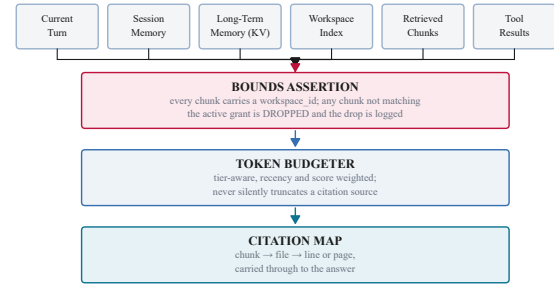


Figure 5: Context assembly. Every retrieved chunk carries the identifier of the workspace it came from; any chunk not matching the active grant is dropped before budgeting, and the drop is recorded.

First match wins, so a deny cannot be argued past

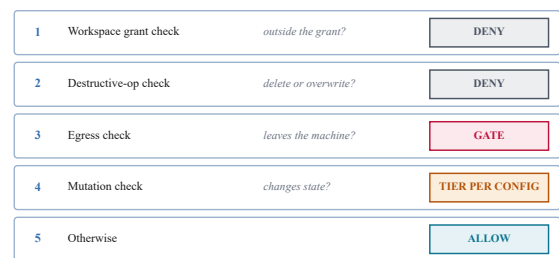


Figure 7: Policy evaluation order. Denials are evaluated before gates, and gates before permissions, so that a refusal cannot be reached around.

Principle P2. Why the destructive tier has no dialogue

There is no approval dialogue for tier three because there is no code path to it. This is what makes the claim that ARTEMIS deletes nothing a property of the system rather than a promise about the model's behaviour. A gate that can be argued past is a policy; a gate that does not exist is an architecture.

Classification proceeds in a fixed order, shown in Figure 7, in which the first matching rule wins. Ordering

the denials first means that no later rule can grant what an earlier rule refused.

Figure 8 works the classification through end to end for a representative request.

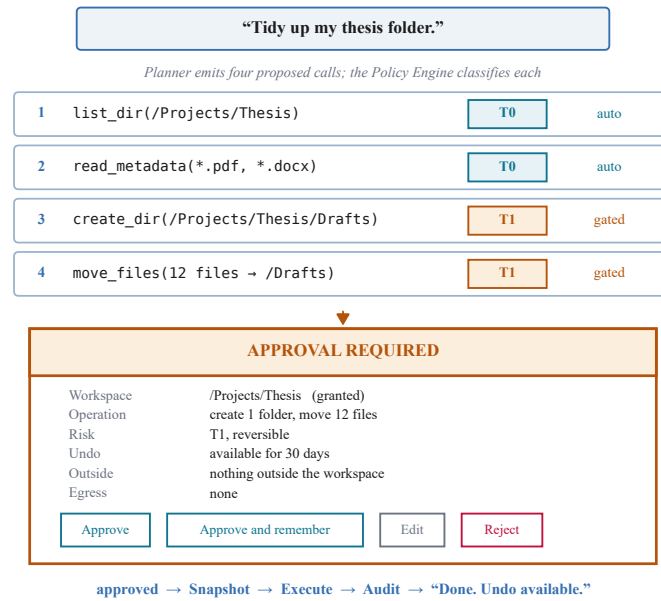


Figure 8: A worked classification and approval. Read operations execute automatically; the two reversible operations are presented together, at the granularity the user actually asked about, with the undo window and the absence of egress stated explicitly.

The option to approve and remember is how the configurable half of the research finding is implemented. Selecting it writes a scoped rule covering this operation type, in this workspace, for this shape of action. It does not lower the user’s protection globally, which is the failure mode that makes permission systems degrade into permanent consent.

5.4 Agent Planner

The planner is built on LangGraph, as recorded in the project’s stack decisions. The architectural constraint on it is simple and absolute: it emits a plan and never executes one. The plan is data, an ordered list of proposed tool calls, and remains inert until the dispatcher acts on it.

The reflection loop, by which the planner revises its approach after observing a result, is bounded by an explicit cycle cap and a wall-clock budget. A planner that cannot converge within those bounds surfaces its partial plan and asks the user, rather than continuing to iterate.

6 Layer 4: Capability Services

The nine features from the project scope map onto capability services. Each is a plugin behind a uniform tool contract, and none touches the filesystem, the network or the database directly. All access passes through the shared substrate shown in Figure 9.

Principle P3. Voice parity is structural

Speech is deliberately not implemented as a capability service. It is normalised at Layer 2 into the same turn object as typed input, and therefore inherits every approval gate automatically. Parity with the text interface is achieved by construction rather than by reimplementing the gates along a second path, which addresses the voice interaction parity theme of the literature review.

6.1 Workspace Broker

The Workspace Broker is the only component that converts a logical reference into a real filesystem path. It is therefore the enforcement point for bounded scope on the write side, and its correctness is load-bearing for the entire trust claim.

The ordering of steps two and three is not incidental. Resolving symbolic links and relative segments before asserting containment is the difference between a boundary and a suggestion; performing the assertion first would allow a path that appears to be inside the grant to resolve to a location outside it.

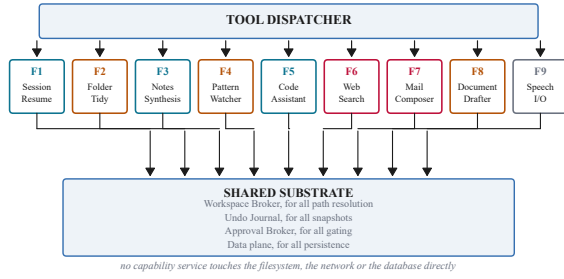


Figure 9: Capability services are reached only through the Tool Dispatcher, and reach storage only through the shared substrate. Each service is coloured by the highest risk tier it may request.

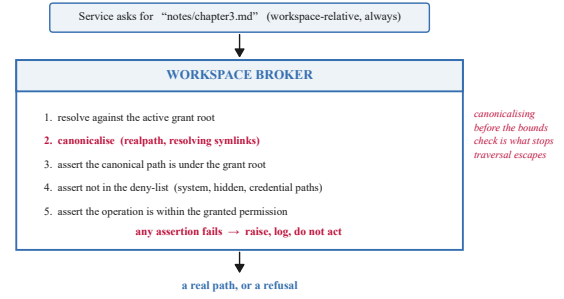


Figure 10: Path resolution and confinement. Canonicalisation occurs before the bounds assertion, which is what prevents symbolic-link and relative-traversal escapes.

Table 3: The nine capability services and the highest risk tier each may request

#	Service	Max tier	Constraint
F1	Session Resume	T0	No model involvement in the first version; a deterministic replay of recorded session state.
F2	Folder Tidy	T1	Moves and groups only. It cannot delete, because no code path exists to do so.
F3	Notes Synthesis	T0	Reads and generates. The output remains a proposal until the drafting service saves it.
F4	Pattern Watcher	T1	Detects three or more repetitions and may only propose an automation rule. It cannot install one.
F5	Code Assistant	T0	Explains, and produces a diff. It never writes to a file and never executes code.
F6	Web Search	T2	On demand only. There is no background browsing, and every query is logged.
F7	Mail Composer	T2	Draft, approve, then send. Sending is gated regardless of user configuration.
F8	Document Drafter	T1	Always writes a new file. It never overwrites an existing one.
F9	Speech Input and Output	n/a	Not a capability service at all, but a Layer 2 modality over identical gates.

6.2 Undo Journal

Before any reversible operation, the Undo Journal records both a forward and an inverse operation, choosing a snapshot strategy according to cost: full content snapshots for small files, copy-on-write links for large ones, and inverse-operation records alone for moves and renames, which need no content copy.

Undo operates at the granularity of a plan rather than of an individual operation. Reversing a folder tidy restores all twelve moves as a single unit and in reverse order, because a plan is the unit the user was asked to approve. Per-operation undo remains available but is secondary, since it does not correspond to anything the user consented to as a whole.

7 Layer 5: The Data Plane

All persistent state is local. The store layout separates the relational system of record, the per-workspace vector indexes, the content-addressed snapshot blobs, a discardable cache, and non-secret configuration. Secrets are held in the operating system keyring and never written into the ARTEMIS directory tree.

Per-workspace index isolation is a deliberate choice rather than an organisational convenience. Because each workspace has its own index, the embeddings of one workspace cannot be retrieved while another is active, even in the presence of a query defect: the index is simply not open. It also reduces workspace revocation to a directory deletion, and it bounds the index rebuild cost that the prototype plan identified as a feasibility risk.

7.1 Memory architecture

The memory design is the clearest differentiator of the system and is shown in Figure 11.

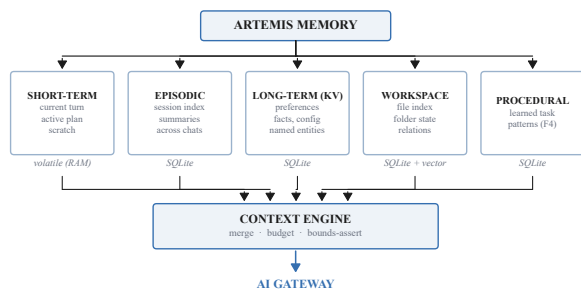
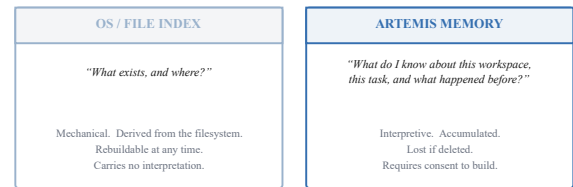


Figure 11: Five kinds of memory, distinguished by lifetime and by store, feeding a single context engine that merges, budgets and bounds-asserts before anything reaches a model.



Conflating the two is the mistake that turns a bounded assistant into a surveillance tool

Figure 12: The file index and ARTEMIS memory answer different questions and have different natures. Conflating them is the design error that turns a bounded assistant into a surveillance tool.

A distinction that the design depends on, and which the final report should state explicitly, is that between the file index and memory proper.

The file index is mechanical. It records what exists and where, is derived from the filesystem, holds paths and content hashes rather than interpretations, and can be rebuilt at any time without loss. Memory is interpretive. It records what the system has learned about a workspace, a task and a user's preferences, accumulates over sessions, and is genuinely lost if deleted. The first requires only a grant; the second requires ongoing consent, and every entry in it is individually visible and individually deletable.

7.2 The disclosure panel

The panel that shows what ARTEMIS currently holds is not a summary the system writes about itself. It is a live query across every store: granted workspaces, indexed files, embedded chunks, remembered facts, retained sessions, automation rules, policy exceptions, actions taken with their undo state, every record of data sent to a cloud provider, and the names of held credentials.

Every row is deletable from the panel, and deletion cascades: removing a workspace removes its index directory, its file rows, its embeddings and its memory entries. This matters because it makes the trust claim falsifiable. A user can check it rather than believe it, which is a stronger position than any assurance the interface could offer.

8 Layer 6: The Remote Access Plane

The remote access plane is drawn outside the core stack because nothing in Layers 1 to 5 depends on it. Removing it entirely leaves a fully functional single-machine system, and that is the test of whether the separation is real rather than notional.

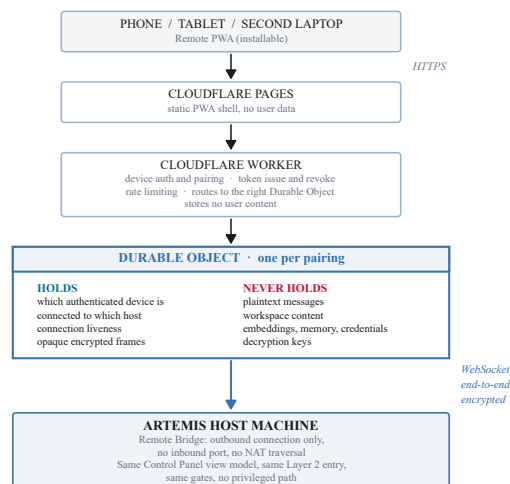


Figure 13: The remote access plane. The durable object records which authenticated device is connected to which host and relays encrypted frames, but holds no plaintext, no workspace content and no keys.

8.1 Pairing

Pairing establishes an end-to-end encrypted channel that the relaying edge cannot read, and requires a confirmation performed physically at the host.

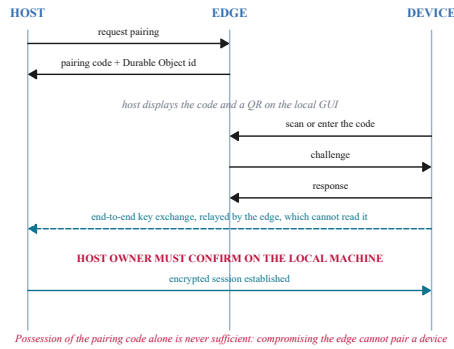


Figure 14: The pairing handshake. The host-side confirmation is deliberate: possession of the pairing code alone is never sufficient, so compromising the edge cannot pair a device.

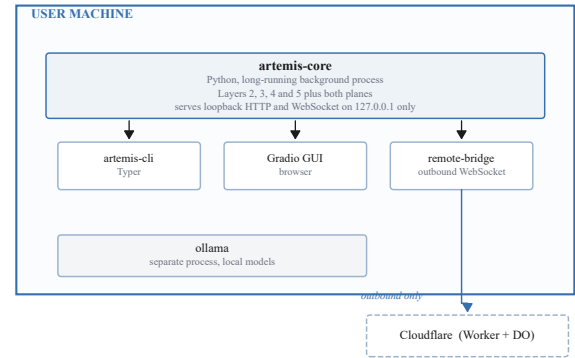


Figure 15: Process topology on the host machine. The core process is the only holder of user data; the interfaces are clients, and the remote bridge dials outward.

8.2 Constraints on remote sessions

The remote plane is a control surface, not a privilege escalation. The following constraints are enforced at the host, not at the edge, because the edge is not trusted.

- A remote session runs at privilege equal to or lower than a local session, never higher.
- A workspace may be marked local-only, in which case it is invisible to remote sessions entirely.
- Outward actions may be configured to require approval at the host regardless of where they were requested. This is enabled by default.
- Sessions expire on a fixed lifetime and on host sleep. There are no indefinite sessions.
- Every remote-originated action is tagged with its originating device in the audit log.
- Revocation is immediate and unilateral from the host: the session is terminated and keys are rotated.
- If the edge is unreachable the host simply has no remote clients, and nothing else about its operation changes.

9 The Security and Trust Plane

Table 4 lists every security control with the component that enforces it and the literature review theme that motivated it. A control with no named enforcement point is an intention rather than a mechanism, which is why the middle column exists.

9.1 Prompt injection

ARTEMIS reads documents, source code and web results, all of which can be influenced by an adversary. The defence is not detection, which is unreliable, but a structural guarantee that retrieved content can never become an authorised instruction.

Principle P4. *The strongest form of the hypothesis*

The fifth defence layer is what makes bounded scope load-bearing rather than rhetorical. Confining the agent to one granted workspace bounds the worst case, not merely the expected case. That is an argument no amount of prompt engineering can make, and it is the clearest evidence this project can offer for its central hypothesis.

9.2 Threat model

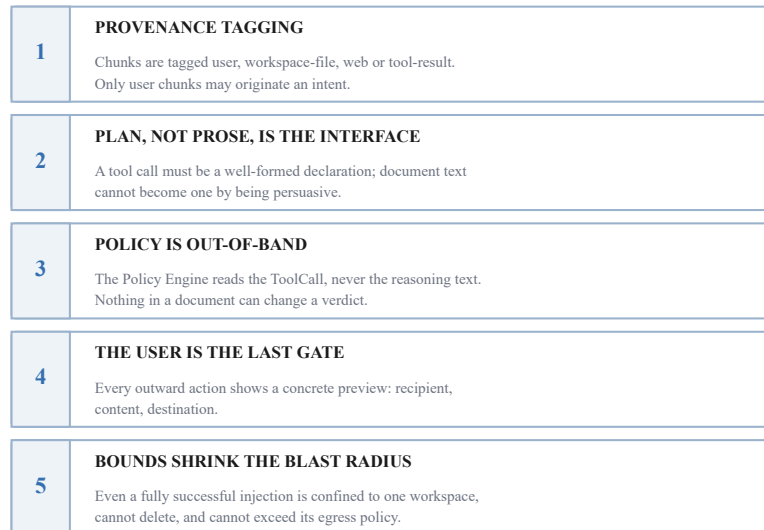
10 The Execution and Performance Plane

The governing rule is that the interactive path stays thin and everything expensive runs asynchronously behind it.

Work is scheduled at five priorities: user-initiated execution, which preempts everything; retrieval for an active turn, which is immediate but budget-capped; incremental index updates for changed files, debounced by roughly two seconds; initial workspace ingestion, which is batched, throttled and resumable; and maintenance,

Table 4: Security controls, their enforcement points, and their research motivation

Control	Enforced at	Research link
Workspace confinement	Workspace Broker	Bounded agents and trust
Explicit grant model	Workspace records and interface	Bounded agents and trust
Tool allowlisting	Tool Dispatcher	Agentic attack surface
Approval gates	Policy Engine	Planning and undo
Risk classification	Policy Engine	Bounded agents and trust
Destructive-operation refusal	Policy tier three, no code path	Bounded agents and trust
Encryption at rest	Encrypted database and keyring	Bounded agents and trust
Encryption in transit	Transport security and end-to-end relay	Bounded agents and trust
Credential vault	Operating system keyring	Bounded agents and trust
Prompt injection defence	Layered, see Section 9.1	Agentic attack surface
Egress control	Model Router egress gate	Privacy and local-first operation
Append-only audit log	Hash-chained log	Assurance frameworks
Single-action undo	Undo Journal	Planning and undo
Remote device authentication	Worker with host confirmation	Bounded agents and trust
Session expiry	Session lifetime and sleep hook	Bounded agents and trust
Least privilege	Remote no greater than local	Bounded agents and trust
Citation grounding	Context Engine citation map	Hallucination and citation accuracy
Human review of code changes	Code assistant emits diffs only	Code assistance with review

**Figure 16:** Five independent layers of prompt injection defence. Each is sufficient to prevent a class of attack, and the final layer bounds the damage of any injection that somehow defeats the preceding four.

which runs only when idle.

Several strategies follow from this structure. Indexing is incremental, so that editing one line re-embeds one chunk rather than an entire file. Retrieval is two-stage, pairing cheap vector recall with a reranking pass, which keeps the index small while preserving quality. The local model is kept warm with an idle-unload timer so that first-token latency does not include a cold load. Every queue is bounded and applies backpressure, so that a saturated system sheds the lowest-priority work visibly rather than falling silently behind. Resource ceilings on memory, index size and processor share are configurable, because a background assistant on a student laptop must never be the reason the machine feels slow.

11 Process and Deployment Topology

Three properties of this arrangement are deliberate. The core process is the only holder of user data, so terminating an interface loses nothing. There is no inbound listening port beyond loopback, because the remote bridge establishes an outbound connection, which means there is no port to scan, no traversal to negotiate and no firewall exception to request. And the local model server runs as a separate process, so that a model crash cannot take down the workspace layer and models can be replaced without touching ARTEMIS itself.

Table 5: Threat model, with mitigations and honestly stated residual risk

Adversary	Capability assumed	Mitigation	Residual risk
Malicious document or web page	Injects instructions into context	The five layers of Section 9.1	The user approves a plausible but harmful action
Compromised model provider	Observes what is sent	Local-first default, per-workspace opt-in, redaction, egress logging	Content sent under explicit opt-in
Compromised edge	Observes relayed traffic and controls the worker	End-to-end encryption, host-side pairing confirmation, no plaintext at the relay	Traffic metadata such as timing and volume
Stolen paired device	Holds a valid session	Session expiry, host revocation, local-only workspaces, host approval for outward actions	Actions taken before revocation
Local attacker with disk access	Reads the ARTEMIS directory	Encryption at rest, keyring for secrets	Attack against a running process's memory
The project team	Operates the telemetry endpoint	No user data is collected by design; logs are schema-fixed and opt-in	None for user content

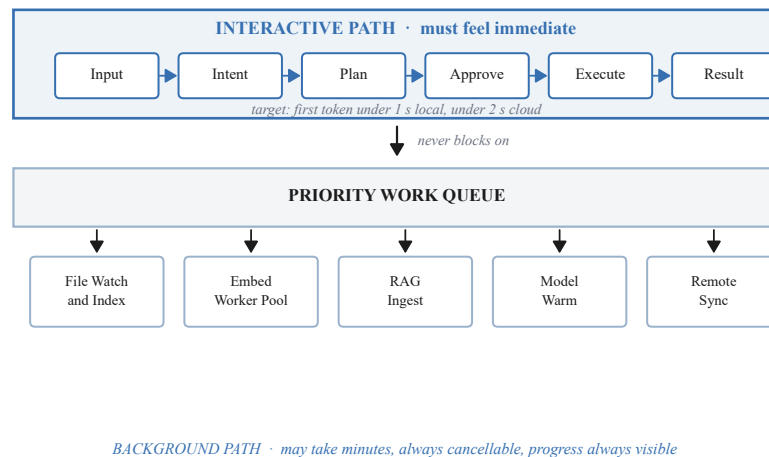


Figure 17: The interactive path never blocks on background work. Background progress is always visible, pausable and cancellable, so that waiting is legible rather than mysterious.

12 Traceability Between Architecture and Research

Table 6 records why each non-obvious structural decision exists. It is the defensible core of the architecture chapter, because it demonstrates that the design follows from the research rather than being justified after the fact.

13 Build Sequence

The staged build sequence from the prototype plan is retained, adjusted so that the two components this revision adds are built early. Both are load-bearing, and both are far cheaper to build before there are features to retrofit them into.

Principle P5. *The most consequential scheduling change*

Stage two now precedes all model work. This means the trust machinery is testable, and demonstrable to a supervisor, before any model non-determinism enters the system. A failing approval gate at that stage is a deterministic bug with a short reproduction, which is not true of the same defect discovered after the planner exists.

14 Open Questions

The prototype plan recorded four feasibility questions, which remain open. Each is restated here with the architectural decision it blocks and the stage by which it must be settled.

Table 6: Each structural decision and the finding that produced it

Architectural decision	Driven by
Workspace Broker as the sole path resolver	The bounded scope hypothesis, and questionnaire results on workspace trust
Policy Engine as a distinct component	The approval-first, configurable finding
Destructive tier with no code path	The requirement that deleting nothing be a property rather than a promise
Undo at plan granularity	Recovery is meaningful at the unit the user approved
Voice normalised at Layer 2	Voice interaction parity, achieved by construction
Citation map through the Context Engine	Hallucination and citation accuracy themes
Code assistant emits diffs only	Code assistance requires human review
Local-first with per-workspace egress opt-in	The local versus cloud split in questionnaire responses
Provenance tagging of context	The agentic attack surface literature
Session resume implemented without a model	Task interruption and resumption; also the lowest-risk first feature
Durable object holds no plaintext	The privacy claim must survive compromise of the edge
Remote privilege no greater than local	Remote convenience must not widen the trust boundary
Background queue with visible progress	Trust requires legibility of what the system is doing

Table 7: Revised build sequence

Stage	Deliverable	Why at this point
1	Workspace foundation: database, Workspace Broker, file watcher and disclosure panel, with no model involvement	The boundary must exist before anything can be bounded
2	Policy Engine, Approval Broker and Undo Journal, exercised with placeholder tools	Cheap now and very expensive to retrofit; provable before any model is introduced
3	AI gateway: Model Router with the local tier and one cloud tier, Context Engine, Agent Planner and Tool Dispatcher	The highest-risk stage, validated against gates that already work
4	Core file features: session resume, folder tidy and pattern watching	The first real capabilities, exercising the read and reversible tiers end to end
5	Understanding features: notes synthesis and code assistance, with the index, retrieval and citation map	Adds the vector index once the boundary is proven
6	Outward features: web search, mail and document drafting	The first outward-tier actions, introduced only after the gates are demonstrated
7	Speech input and output over the existing Turn Normaliser	Should require no new gates. If it does, Layer 2 was designed wrongly, which makes this stage the test of the parity claim
8	Remote access plane	Genuinely optional; a complete product exists without it

Table 8: Open questions carried forward, with what each one blocks

Question	Blocks	By stage
Which local model size is realistic on a typical student laptop?	The default local tier, and the resource envelope	3
Which desktop interface approach is most reliable across features?	Layer 1. The current inclination is the local webview, since one view model can then serve both the local and remote interfaces	1
Are local speech models accurate enough to avoid cloud fallback?	Whether voice input ever becomes an outward-tier action	7
How large can a workspace index grow before retrieval or rebuild degrades?	Per-workspace index sizing and the two-stage retrieval thresholds	5

This revision raises three further questions. Whether encryption at rest should use an encrypted database or rely on operating-system disk encryption together with the keyring, which determines how the encryption claim in Table 4 may be stated. Whether a reranking model is affordable locally, or whether vector-only retrieval is sufficient, which determines the two-stage retrieval design. And whether the relay needs message durability at all, since a live-only channel is both simpler and a stronger form of the claim that the relay holds no data.

15 Summary of Changes from Revision 1

Table 9: Changes from Revision 1, with rationale

#	Change	Rationale
1	The AI gateway is decomposed into router, context, policy and planner components behind one dispatcher	The single-component design was untestable and difficult to defend
2	A Policy Engine is introduced as a first-class component with four risk tiers	Converts the approval-first finding into enforced structure
3	Security is promoted from a component to a cross-cutting plane	Security is a property of every interaction, not a stage
4	Execution and performance is added as a second cross-cutting plane	Makes the synchronous and asynchronous split explicit and testable
5	The remote plane is drawn outside the core stack	Demonstrates the privacy claim: removing it changes nothing locally
6	Memory is formally separated from the file index	The clearest differentiator, and the clearest privacy boundary
7	A Turn Normaliser is introduced at Layer 2	Makes voice parity structural rather than duplicated
8	The Workspace Broker becomes the sole path resolver	Turns bounded scope from a policy into an invariant
9	Undo moves to plan granularity	Matches the unit the user was asked to approve
10	Prompt injection is addressed as an explicit five-layer defence	Answers the central threat identified in the review
11	The build sequence is reordered to place trust machinery before model work	Cheap now, nearly impossible to retrofit later
12	A traceability matrix is added	Ties every structural choice to a cited finding